

EigenpackConvert Documentation

Ryan Hill
Ryan.Hill@ed.ac.uk

January 31, 2024

Contents

1	File Format Specification	2
1.1	Overview	2
1.2	Float Compression	3
1.3	Specification Details	3
1.3.1	Metadata	3
1.3.2	Compressed File	4
1.3.3	BasisVectors	5
1.3.4	LatticeFermion	5
1.3.5	CompressedBasisVectors	5
1.3.6	CompressedLatticeFermion	5
1.3.7	CompressedFloats<size>	5
1.3.8	Coefficients	6
2	Program Specification	7
2.1	Overview	7
3	Program Compilation and Running	9
3.1	Compilation	9
3.2	Running	10
3.3	Potential Issues	10
3.3.1	Insufficient MPI Ranks	10
3.3.2	Non-factorisable local volumes	10

0	1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16	17
18	19	20	21	22	23	24	25	26
27	28	29	30	31	32	33	34	35
36	37	38	39	40	41	42	43	44
45	46	47	48	49	50	51	52	53
54	55	56	57	58	59	60	61	62
63	64	65	66	67	68	69	70	71
72	73	74	75	76	77	78	79	80

Figure 1: Allocation of fine sites to coarse sites in the GPT Eigenpack.

1 File Format Specification

1.1 Overview

The GPT Eigenpack multifile format describes lattice eigenvectors generated under the “Local Coherence” scheme. Under this scheme, the lattice (or as we will describe it going forwards, the **fine grid**) is coarsened into a **coarse grid**, as exemplified for 2D in Figure 1.1 with a coarsening of 3 in both the x and y dimensions. In Figure 1.1, sites 0, 1, 2, 9, 10, 11, 18, 19, and 20 are associated with the coarse site 0, with other associations illustrated by the figure also. Data is similarly associated (but much harder to draw) for the 4D or 5D lattice data we are interested in. For each **fine site**, a limited number of eigenvectors are stored. We will refer to these as **basis vectors**. Typically, a substantially larger number of eigenvectors are then expressed as a **set of coefficients** of these **basis vectors** on each **coarse site**. The larger set of eigenvectors can then be generated per **fine site** by taking the dot product of the **fine site**’s set of **basis vectors** with the **coarse coefficients** for that **fine site**’s corresponding **coarse site**. Since the eigenvectors of similarly-situated lattice sites are highly correlated (or “**locally coherent**”), this is a good approximation of the true set of eigenvectors per **fine site**. We therefore have two pieces of data to keep track of: **basis vectors** for each **fine site**, and **coarse coefficients** for each **coarse site**.

The **block** is the organisational unit of the GPT Eigenpack format. A **block** comprises the set of **coarse coefficients** for a particular **coarse site** in addition to the **basis vectors** for each **fine site** covered by the **coarse site**. These **blocks** are then grouped into **volumes**, where (similarly to how **fine sites** are grouped into **blocks**) each **volume** covers a consistent number of **blocks** in

each dimension. Each **volume** is stored as a separate **file**. This makes the GPT Eigenpack highly efficient to load when the local volume of the compute nodes exactly factorises into the volumes covered by each Eigenpack **file**, since this permits perfect parallelism where two nodes will never need to access the same **file** during I/O. These **files** are also equally distributed between 32 **directories**.

1.2 Float Compression

In addition to the compression obtained by using the Local Coherence scheme for storing eigenvectors, a further step of compression is supported by allowing the least significant basis vectors to be stored with reduced precision. This “reduced precision” format is defined in terms of

1.3 Specification Details

1.3.1 Metadata

The Metadata file is an ASCII plaintext file with key-value pairs given in format

$$\langle \text{key} \rangle = \langle \text{value} \rangle$$

Tabulated below are a list of identifiers and a variable name used in subsequent sections to refer to the value of this identifier. All values are unsigned integers.

Name	Variable Name	Description
s[0]		The total number of fine sites in the X dimension contained in each eigenvector file.
s[1]		The total number of fine sites in the Y dimension contained in each eigenvector file.
s[2]		The total number of fine sites in the Z dimension contained in each eigenvector file.
s[3]		The total number of fine sites in the T dimension contained in each eigenvector file.
s[4]		The total number of fine sites in the Ls dimension contained in each eigenvector file.
b[0]	x	The total number of fine sites in the X dimension contained in each block.
b[1]	y	The total number of fine sites in the Y dimension contained in each block.
b[2]	z	The total number of fine sites in the Z dimension contained in each block.
b[3]	t	The total number of fine sites in the T dimension contained in each block.
b[4]	Ls	The total number of fine sites in the Ls dimension contained in each block.

nb[0]	xBlock	The total number of blocks in the X dimension contained in each eigenvector file. This is equal to $s[0]/b[0]$.
nb[1]	yBlock	The total number of blocks in the Y dimension contained in each eigenvector file. This is equal to $s[1]/b[1]$.
nb[2]	zBlock	The total number of blocks in the Z dimension contained in each eigenvector file. This is equal to $s[2]/b[2]$.
nb[3]	tBlock	The total number of blocks in the T dimension contained in each eigenvector file. This is equal to $s[3]/b[3]$.
nb[4]	LsBlock	The total number of blocks in the Ls dimension contained in each eigenvector file. This is equal to $s[4]/b[4]$.
neig	eigenvectorCount	The total number of eigenvectors stored as coefficients of coarse grid sites.
nkeep_single	fp32BasisCount	The number of basis vectors on each fine site stored in single precision.
nkeep		The total number of basis vectors on each fine site. The number of compressed basis vectors on each fine site (referred to as fp16BasisCount below) is defined by $n_{\text{keep}} - n_{\text{keep}}^{\text{single}}$.
blocks		The total number of blocks stored per file. This is equal to $nb[0]*nb[1]*nb[2]*nb[3]*nb[4]$.
FP16_COEF_EXP_SHARE_FLOATS		The number of eigenvector coefficients for the compressed basis vectors that are compressed using the same scaling factor.
crc32[i]		The crc32 checksum for data file i .

1.3.2 Compressed File

Name	Type	Description
basisFP32	BasisVectors[LsBlock][tBlock][zBlock][yBlock][xBlock][fp32BasisCount]	A list of basis vectors stored in single precision for each block contained within the file.
basisFP16	CompressedBasisVectors[LsBlock][tBlock][zBlock][yBlock][xBlock][fp16BasisCount]	A list of additional basis vectors stored in a compressed format for each block contained within the file.
coefficients	Coefficients[eigenvectorCount][LsBlock][tBlock][zBlock][yBlock][xBlock]	A list of coefficients used to reconstruct eigenvectors by taking their product with the basis vectors. One set of coefficients is stored per eigenvector per block.

1.3.3 BasisVectors

Name	Type	Description
sites	LatticeFermion[Ls][t][z][y][x]	One Basis Vector on each fine lattice site in the given block. The five dimensions given in the type are the extent of the fine grid that a block covers.

1.3.4 LatticeFermion

Name	Type	Description
elements	float[4][3][2]	Elements of a SpinColourMatrix. The three dimensions are Spin, Colour, and complex components.

1.3.5 CompressedBasisVectors

Name	Type	Description
sites	CompressedLatticeFermion[Ls][t][z][y][x]	Compressed data for one Basis Vector on each fine lattice site in the given block. The five dimensions given in the type are the extent of the fine grid that a block covers.

1.3.6 CompressedLatticeFermion

Name	Type	Description
elements	CompressedFloats<24>	Elements of a SpinColourMatrix. The elements are arranged in the same order as the LatticeFermion elements.

1.3.7 CompressedFloats<size>

Name	Type	Description
exponent	uint16	The common scale for all elements.
elements	int16[size]	Two bytes per compressed float held by the struct. Refer to ?? to see how to compress and decompress the elements of this format to floats.

1.3.8 Coefficients

Name	Type	Description
fp32Coefs	float[fp32BasisCount][2]	Coefficients for the uncompressed basis vectors. Each coefficient is complex and is represented by two floats.
fp16Coefs	CompressedFloats<10> [2*fp16BasisCount/10]	Coefficients for the compressed basis vectors. The coefficients are stored in chunks of 10, with each chunk stored in a CompressedFloats<10> struct. Each coefficient is complex and is represented by two floats.

2 Program Specification

2.1 Overview

This program is designed to convert a GPT Eigenpack into a Scidac Eigenpack, using Grid and Hadrons to define the Scidac Eigenpack data structure and to write it to disk.

Note: The program will only currently produce correct results if the volumes contained in the input files exactly factorise into the local volume for each MPI process. In other words, each file must be loaded and fully parsed by exactly one MPI process.

The organisation of the header files in use (and the two entry points, `main.cpp` and `Python.cpp`) are shown in figure 2.1.

The main class responsible for the operation of the compiled binary is `GridEigenpackReader`, which is defined in `EigenpackReader/GridEigenpackReader.hpp`. This class carries out the high-level operations of dictating which sections of a file to read and then where to insert this into the Scidac eigenpack in a batched fashion in order to stay within memory limits, as well as converting the eigenvalues. To do this, it relies heavily on the `Metadata` class defined in `EigenpackReader/Metadata/Metadata.hpp` to define the data layout it expects to find in the eigenpack files. These two classes constitute most of the GPT conversion logic, with the remaining classes mostly serving the role of various utilities or servicing minor functions and operations. Throughout the rest of this section we will cover the definitions of the objects in the codebase and how they work together to perform the conversion.

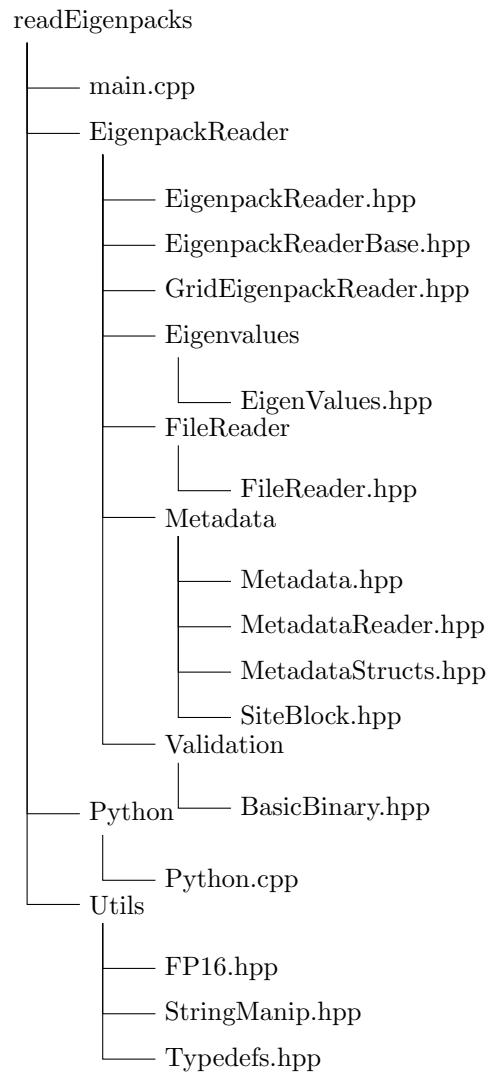


Figure 2: Organisation of the source code.

3 Program Compilation and Running

3.1 Compilation

- Run autoreconf:
`./bootstrap.sh.`
- Create an out-of-source build directory:
`mkdir build; cd build`
- Configure with the following options:
`../configure`
`--prefix=<path/to/install/program>`
`--with-grid=<path/to/grid>`
`--with-hadrons=<path/to/hadrons>`
`CC=clang` # Whichever compiler was used for
Grid & Hadrons
`CXX=clang++` # Whichever compiler was used for
Grid & Hadrons
`nbasis=250` # Number of basis (fine) vectors. Must
exactly equal the number of basis vec-
tors used by the eigenpack.
`concbasis=2` # Number of basis (fine) vectors to
convert per batch. Low numbers harm
performance, but use less memory.
`conceigen=100` # Number of (coarse) eigenvectors to
convert per batch. Low numbers harm
performance, but use less memory.
- Make and install
`make`
`make install`

3.2 Running

The program can be ran with the following options:

```
path/to/EigenpackConverter
--grid 64.64.64.128      # Fine lattice size
--mpi 1.1.2.2            # MPI layout for the conversion
--Ls 12                  # Lattice Ls extent
--config 1000            # Configuration number
--i /path/to/input       # Path to GPT eigenpack (containing meta-
                        data.txt)
--o /path/to/output      # Directory to output Scidac eigenpack to
--outmode <fmt>          # Which format to output data in:
```

- none: No output
- basic: Debug format, do not use
- hadrons: Scidac format

3.3 Potential Issues

3.3.1 Insufficient MPI Ranks

MPI processes can write a maximum of 2^{32} bytes each. Since Scidac eigen-vector files typically exceed this, multiple MPI processes are usually needed to successfully write an eigenpack. For example, an eigenpack with eigenvectors that decompress to 18 GiB must be written using at least 9 MPI processes. The program will check the number of MPI processes in use before performing the conversion, and exit if an insufficient number is detected. Typically, enough MPI processes will be usable on a single node to convert an eigenpack.

3.3.2 Non-factorisable local volumes

Each file of the GPT eigenpack files corresponds to a contiguous chunk of the lattice. This is given by the ‘s’ parameter in the eigenpack metadata file. The dimensions of this chunk must exactly factorise into the local volume of the grid used to perform the conversion due to current limitations of the conversion implementation. For example, an eigenpack with dimensions of $4 \times 3 \times 4 \times 4$ can only be converted on a grid with a local volume that is a multiple of 4 in the x, z, and t dimensions, and a multiple of 3 in the y dimension. The program will exit with an error message if this condition is not fulfilled.